Plume XPU Console 1.2.0 and 1.3.0 — Technical Note PXC-TN-01

Nov 2, 2026 · @Russell Smithson

1. Abstract and document control

Releases 1.2.0 and 1.3.0 of the Plume XPU Console implement ByteFoundry as a virtual XPU. A deterministic cellular law runs over a 128 × 128 frame of 160-bit cells, in two residue rings in sequence: $Z/2^{20}$ (stage BF1) and $Z/5^{8}$ (stage BF2). Each stage is written once in the array language LCUF-A/0.3 and executed bit-identically by three backends: single-thread WebAssembly, multi-threaded WebAssembly over shared memory, and WebGPU. Every intermediate state is sealed as a RUSS 1.3 container whose RUSSL/1.0 program declares a 20-layer 2-adic pyramid.

Release 1.2.0 makes the whole verification path independent of Python: the RUSSL reference lowering and the corpus schema subset are ported to JavaScript and agree with the Python reference on every test case. Release 1.3.0 raises single-thread BF1 throughput from about 35 to about 80 M lane-updates/s on the build host, by vectorising the Life rule. It also adds a live presentation whose figures are measured on the viewer's machine against a plain-JavaScript baseline.

Field	Value
Identifier	PXC-TN-01, revision 1
Releases covered	1.2.0 (commit e779105, Blank-MIMIC-CIMIM) · 1.3.0 (commit 958485e, MIMIC-CIMIM-Demonstration)
Status	Technical record of released software; not externally reviewed or certified
Copyright	Copyright (C) 2026 RUSSELL PHILIP SMITHSON
Licence	GPL-3.0-only (both repositories)
Language versions	LCUF-A/0.3 (compiler 0.3.1 in 1.2.0, 0.3.2 in 1.3.0) · RUSSL/1.0 · RUSS 1.3 · BFTL/1

Field	Value
Evidence basis	Automated suites run on the build host, two independent code reviews, and an operator run on an AMD GCN-5 (Vega) GPU with 8 CPU threads

Conventions: "lane-update" is one 20-bit word of one lane of one cell advanced by one step; a frame step is 131,072 lane-updates. M/s means millions of lane-updates per second. Normative words (MUST, MUST NOT) appear only where the software enforces the rule.

2. Scope and claims discipline

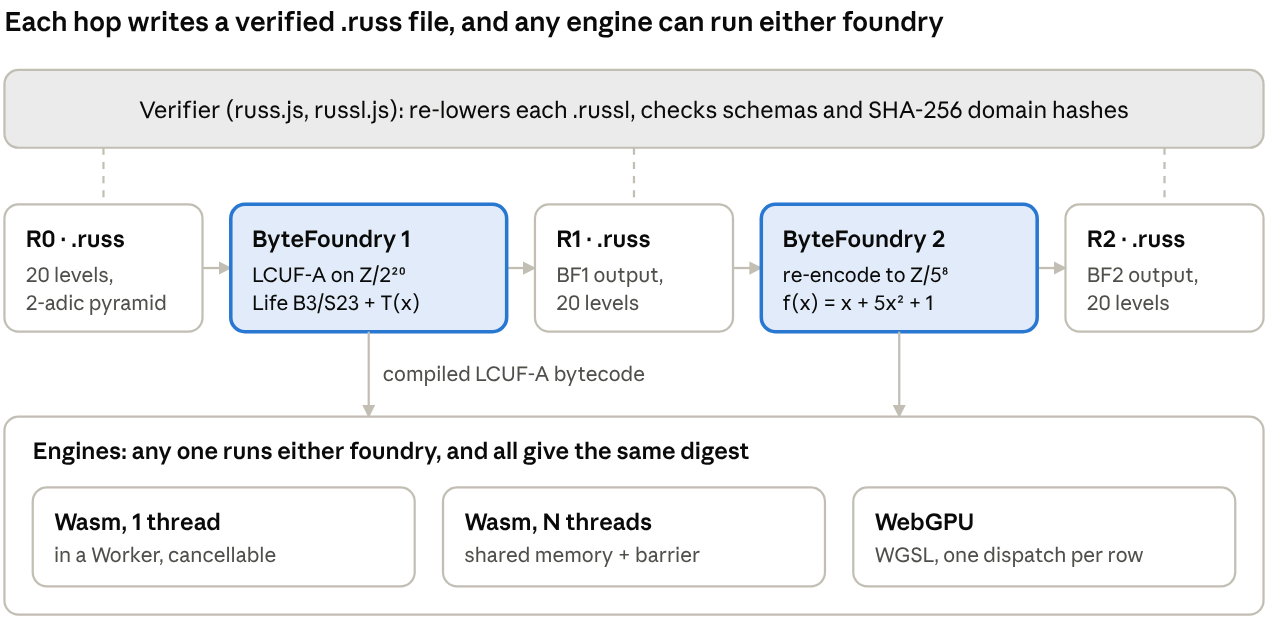
This note records what the two releases establish, the evidence for each claim, and the limits of that evidence. Each claim below is marked with the strongest evidence that supports it.

Claim	Evidence	Section
BF1 and BF2 are bit-exact against independent NumPy implementations of the same laws	Automated test, 40 steps per stage and the BF1 → BF2 chain	11
The three backends produce byte-identical frames	Automated test (CPU engines); browser test on a software GPU; operator run on AMD GCN-5 with matching tuple ids	6, 11
$T(x) = x + (x^2 \vee 5)$ and $f(x) = x + 5x^2 + 1$ are single-cycle on $Z/2^{20}$ and $Z/5^8$	Exhaustive iteration (2^{20} and 390,625 states)	4
Reduction mod 2^j commutes with the BF1 step (hologram property)	Proof sketch; tested at j = 1, 2, 4, 8, 12, 13, 16, 20	4, 11
JS lowering of RUSSL equals the Python reference lowering	Byte-identical canonical IR on 15 valid sources; same error code on 13 invalid cases	8
The JS container verifier refuses what the Python verifier refuses	15 crafted containers refused by both	8
1.3.0 is faster on the CPU than 1.2.0, with unchanged results	Benchmarks on the build host; all suites unchanged	10

The releases do not claim:

- Any physical, quantum or optical effect. "Hologram" names an exact algebraic property (Section 4.5), and "plume" names the set of bits on which two runs differ.
- Equivalence to, or competitiveness with, dedicated hardware accelerators. "XPU" is the role ByteFoundry plays in the pipeline, and implies no silicon.
- Conformance to the full RUSSL corpus. The corpus records 293 requirements, of which 259 are NOT_RUN. These releases implement the reference-profile parser, lowering and container checks only.
- Portable performance figures. Throughput depends on the CPU, the GPU, the browser and the build. Every figure in this note names its machine.

3. Architecture



Plume XPU Console pipeline, releases 1.2.0 and 1.3.0 · 3 files, 2 foundries, 3 engines

The console is a three-file pipeline with two compute stages between the files. R0 is generated from a seed as a 20-level 2-adic pyramid and written as a RUSS 1.3 container whose RUSSL/1.0 program records how it was made. ByteFoundry 1 runs the BF1 program in the 2-adic ring and writes R1; ByteFoundry 2 re-encodes R1 into the 5-adic ring, runs the BF2 law and writes R2. Each foundry is an LCUF-A/0.3 program compiled once to five-word bytecode; the same bytecode is executed by whichever engine the operator selects, and the three engines are required to agree bit for bit. No file is passed to the next stage until the verifier has accepted it, and the verifier is the same code whether a file was produced in this session or loaded from disk.

The layers have separate responsibilities. The **container layer** (`russ.js`, `russl.js`, `schema.js`) owns bytes, hashes and the RUSSL program, and never runs a law. The **language layer** (`lcufa.js`) owns parsing, validation, compilation and the reference interpreter, and never touches a file. The **engine layer** (`xpu.c` compiled to two Wasm modules, the WGSL kernel, `engines.js`) owns execution, threads, cancellation and timing, and never interprets source. The page (`page.html`) only wires these together, so each layer can be tested without the others (§12).

4. Mathematical foundations

4.1 State space

A frame is a function $P : \Lambda \times K \to R$. $\Lambda = (\mathbb{Z}/128)^2$ is a toroidal lattice of 16,384 cells, $K = \{0, \ldots, 7\}$ indexes the eight lanes of a cell, and R is the stage ring: $\mathbb{Z}/2^{20}$ for BF1, $\mathbb{Z}/5^8$ for BF2. A cell holds $8 \times 20 = 160$ bits ("Pixel160"), and a lane holds one Byte20 word. Digit 0 of each lane, the bit-0 plane, carries a Life computation. Digits 1 to n−1 carry the plume field. $N(p)$ denotes the eight Moore neighbours of p on the torus.

4.2 The BF1 law in $\mathbb{Z}/2^{20}$

$T(x) = x + (x^2 \vee C)$ is the Klimov–Shamir mapping [1]. It is a single-cycle permutation of $\mathbb{Z}/2^n$ exactly when bits 0 and 2 of C are both 1; $C = 5$ (binary 101) satisfies this. BF1 advances each lane k of each cell p as follows:

```
S_k(p) = \sum_{q \in N(p)} P_{(k+1) \bmod 8}(q) \pmod{2^{20}}, \qquad
\Phi_k(p) = T(P_k(p)) \oplus S_k(p)
```

```
P'_k(p) = L_{B,S}\big(P^{(0)}_k\big)(p) + 2\,\Phi_k(p) \pmod{2^{20}}
```

L_{B,S} is the outer-totalistic Life rule (default B3/S23) applied to digit 0 of lane k. Exhaustive iteration from 0 confirms that T has period $1{,}048{,}576 = 2^{20}$.

4.3 The BF2 law in $\mathbb{Z}/5^8$

No ring homomorphism exists between $\mathbb{Z}_2$ and $\mathbb{Z}_5$, so BF2 uses a 5-adic analogue rather than a translation. The chosen permutation polynomial is $f(x) = x + 5x^2 + 1$, with $\boxplus$ denoting digit-wise addition mod 5 without carries (the 5-adic analogue of XOR):

```
P'_k(p) = L_{B,S}\big([P_k \bmod 5 \neq 0]\big)(p) + 5\,\big(f(P_k(p))
\boxplus S_k(p)\big) \pmod{5^{8}}
```

Larin [2] gives a complete description of the polynomials that are transitive modulo m. The coefficient pattern used here ($a \equiv 0$, $b \equiv 1$, $c \not\equiv 0 \bmod p$) was chosen as a candidate, but this note relies on direct computation rather than on the criterion: iterating f from 0 visits all 390,625 residues before returning. A randomised check of 20,000 pairs confirms that f is 1-Lipschitz in the 5-adic metric.

4.4 Precision ordering

Every operation in either law is a T-function of the neighbourhood: output digit i depends only on input digits 0 to i of the cell and its neighbours. Equivalently, the step F is 1-Lipschitz in the p-adic metric:

```
|F(x) - F(y)|_p \le |x - y|_p
```

The LCUF-A operation set of Section 5 is closed under this property. Addition, multiplication, constant OR (p = 2), digit-wise addition, multiplication by p, lane rotation, neighbour sums, reduction mod p^j and the Life rule on digit 0 are each 1-Lipschitz. Therefore every admissible LCUF-A program in a given ring inherits the properties below.

4.5 Hologram theorem

Let π_j be reduction mod p^j. For every $1 \leqslant j \leqslant n$ and every step count t:

```
\pi_j \circ F_n^{\,t} = F_j^{\,t} \circ \pi_j
```

Running the law at j digits therefore gives exactly the low j digits of the full-precision run. In 1.2.0 and later, each R file stores the SHA-256 of all 20 reductions (state/pyramid.json). The console's Check hologram reproduces layer j of R1 from layer j of R0 alone, at j = 1, 2, 4, 8, 12, 16 and 20.

4.6 Cone bound

Let $D_t = F^t(P) - F^t(P')$, where P and P′ differ only at cell p_0. Because each step reads only the Moore neighbourhood, $D_t(p) \neq 0$ implies $\|p - p_0\|_\infty \leqslant t$ on the torus. The live presentation checks this inequality at every step.

4.7 Floor-rise lemma

Suppose two frames agree on digit 0 everywhere and first differ at digit $i \geqslant 1$. Then after one step they first differ at digit i + 1 or higher. The Life term depends only on digit 0, and the remaining term is multiplied by p, which moves every difference up one place. A difference that never reaches digit 0 therefore vanishes within n − i steps. Observed on the build host: a flip at bit 3 produced lowest differing bits 4, 5, 6, …, 11 over eight steps. Only a difference on digit 0, which changes the Life computation, can persist.

4.8 Echo lemma

Digit i of P′ depends only on digits below i of P and on digit 0. By induction, after n − 1 steps the upper digits no longer depend on the initial upper digits at all. They are a deterministic function of the last n − 1 Life generations in the light cone. Where the Life plane is quiet, the field settles to a uniform value; this was observed as identical words (30f96 in every lane) in an empty region. Where Life is active, the upper digits show its history.

4.9 Cross-ring re-encoding

BF2 maps a 20-bit word into $Z/5^8$ and back with the LCUF-A operations from2 and to2:

```
\mathrm{from2}(x) = (x \bmod 2) + 5\,\big(\lfloor x/2 \rfloor \bmod
5^{7}\big), \qquad \mathrm{to2}(v) = \big((v \bmod 5) \bmod 2\big) +
2\,\lfloor v/5 \rfloor
```

The encoding preserves the Life bit exactly. It discards the part of $\lfloor x/2 \rfloor$ at or above 5^7 = 78,125, about 2.7 of the 19 upper bits. This loss is a property of crossing between the two rings, documented in both releases. The R2 pyramid is a 2-adic pyramid of the decoded field.

5. LCUF-A/0.3: the array language

LCUF-A is the array profile of the LCUF column-language family. In this pipeline it plays the role NumPy plays for Python: each row names one whole-frame operation, and a native backend runs that operation over all 131,072 words. The scalar language LCUF/0.1 (six operations on i64, bool and text) and the LCUF-X/0.2 host kernels are unchanged. LCUF-A adds frames as values.

5.1 Concrete syntax

```
LCUF-A/0.3
module <name>                                  module = [A-Za-z_][A-Za-z0-9_.-]*
ring p=<prime> n=<digits>                      decimal, no leading zeros, n >= 1,
p^n <= 2^21; p = 2 requires n = 20
frame lanes=8 height=128 width=128
phase | id | op | inputs | output | arg
<phase> | <id> | <op> | <in>[,<in>] | <out> | <arg> | -
end
```

Blank lines and lines beginning with # are ignored. Phases appear in the order init, step, final: init runs once, step runs t times, final runs once. The register in holds the input frame, and the program MUST write out.

5.2 Operations

Op	Code	Arity	Arg	Semantics (all results mod p^n)
copy	1	1	—	o = a
life	2	1	—	Outer-totalistic B/S rule on digit 0 of each lane; alive ⇔ a mod p ≠ 0; result 0 or 1
rot	3	1	c ∈ 0..7	o[cell, k] = a[cell, (k + c) mod 8]
nsum	4	1	—	Sum of the 8 Moore neighbours, same lane, toroidal
sq	5	1	—	a^2
mul	6	2	—	a · b
mulc	7	1	c	a · (c mod p^n)
add	8	2	—	a + b
addc	9	1	c	a + (c mod p^n)
orc	10	1	c	a ∨ (c mod p^n); p = 2 only
dadd	11	2	—	Digit-wise addition mod p without carries (XOR when p = 2)
shift	12	1	—	a · p
from2	13	1	—	Re-encode a 20-bit word (Section 4.9); identity mask for p = 2
to2	14	1	—	Decode to a 20-bit word; identity for p = 2
modj	15	1	c ∈ 1..n	a mod p^c (pyramid level c)

5.3 Static admission

The compiler refuses a program, with a line-numbered diagnostic, when any of these holds:

- The header, ring or frame line is malformed; p is not prime; n < 1; $p^n > 2^{21}$; or p = 2 with n ≠ 20.
- An operation is unknown, its arity is wrong, an argument is missing or is longer than 10 digits, or an argument is out of range (rot > 7, modj outside 1..n, orc with p ≠ 2).
- A row id repeats; a name is malformed or reserved (**proto**, constructor, prototype, undefined, null, hasOwnProperty, toString, valueOf); or an output is in or equals one of its own inputs.
- A name is read before it is written. A step row may read only init values or rows above it in the same iteration.
- out is never written; more than 16 registers are needed; more than 256 rows; or the source exceeds 65,536 characters.

The output-differs-from-inputs rule exists for the GPU, which cannot bind a written buffer as an input in the same dispatch. Constants for addc, mulc and orc are reduced mod p^n at compile time, and again inside each engine. Without this, the C and WGSL engines wrap at 2^{32} while the reference interpreter does not; this was a 1.1.0 defect.

5.4 Bytecode

A compiled program is a Uint32Array of five words per row (op, out, a, b, c) and a register map. Registers are assigned in first-definition order, with in at slot 0. The phase counts (ninit, nstep, nfinal) travel with the code, and the same bytecode drives all three backends.

5.5 The two stage programs

BF1 (bytefoundry.bf1.plume2) has 12 rows and uses 12 registers. Its step phase is life, rot 1, nsum, sq, orc 5, add, dadd, shift, add, copy. BF2 (bytefoundry.bf2.plume5) has 13 rows and uses 13 registers, replacing orc 5 with mulc 5 and addc 1 to realise $x + 5x^2 + 1$. Each program's SHA-256 is recorded in the provenance of every R file it produces.

6. Execution engines

All backends implement one interface: run(compiled, input, steps, rule) returns a promise of { out, ms }. Requests are serialised per engine through a promise queue, so a second call waits for the first; before the queue existed, two overlapping calls could corrupt

shared state (a 1.1.0 defect). Before any backend runs, a shared admission check enforces three things: the program is compiled, the input is exactly 131,072 words each below 2^{20}, and the step count is an integer from 1 to 100,000.

6.1 Native engine (src/xpu.c)

The engine is freestanding C (no libc) compiled with clang 18 for wasm32 at -O3 with SIMD128. Two builds come from one source:

Build	Memory	1.2.0 size	1.3.0 size
xpu.wasm (single thread)	20 MiB, exported	7,272 B	11,814 B
xpu-mt.wasm (threaded)	20 MiB shared, imported; atomics; bulk memory	7,889 B	11,780 B

Linear memory holds 16 frame registers of 512 KiB each in chunky order: index = (y · 128 + x) · 8 + lane. It also holds the instruction buffer (256 rows × 5 words), a 64 × 32 KiB block of per-worker stacks, the barrier counters and a progress counter. A reviewer checked the layout on a live 1.1.0 instance (later releases add only an 8 KiB digit table and two counters): registers end at 8,394,800, the worker stacks at 10,491,952 and the main stack at 10,557,504, all within 20 MiB with no overlap.

The engine validates its own inputs rather than trusting the compiler:

- configure(p, n, birth, survive) returns 0 unless p is prime, $1 \leqslant n \leqslant 21$, $p^n \leqslant 2^{21}$, and n = 20 when p = 2.
- load_program returns 0 on an unknown op, an out-of-range register, an output aliasing an input, a rot or modj argument out of range, or a phase count that would wrap in u32.
- run returns 0 when nt = 0, nt > 128 or tid ⩾ nt.

6.2 Bit-exactness rules

Every operation uses u32 arithmetic with explicit reductions, so C, WGSL and the JS reference agree on every word. In $Z/2^{20}$, mod is a mask, multiplication is wrapped u32 multiplication then a mask, and dadd is XOR. In odd rings with $M \leqslant 2^{21}$, multiplication splits one operand into 10-bit chunks so that no intermediate exceeds 2^{32}:

```
a \cdot b \bmod M = \Big( \big( (a \cdot \lfloor b/2^{10} \rfloor \bmod M)
\cdot 2^{10} \big) \bmod M + (a \cdot (b \bmod 2^{10})) \bmod M \Big) \bmod M
```

The JS reference interpreter computes the same values independently, with BigInt for products and exact Number arithmetic elsewhere.

6.3 Threading and synchronization

The threaded build runs in N workers (2 ⩽ N ⩽ 16, from navigator.hardwareConcurrency) sharing one WebAssembly.Memory. Worker t owns lattice rows [t·128/N, (t+1)·128/N) and sets its own __stack_pointer from stack_top(t). The barrier is generation-counted: the last arrival resets the count, increments the generation and calls memory.atomic.notify, and the others wait with memory.atomic.wait32 on the generation word. Shared memory requires cross-origin isolation, so the bundled server sends Cross-Origin-Opener-Policy: same-origin and Cross-Origin-Embedder-Policy: credentialless.

6.4 Liveness: watchdog and cancellation

Worker 0 increments a progress counter in shared memory after every barrier. The host reads it every 500 ms and stops a run only when it has not moved for 30 s. This replaced the 1.1.0 wall-clock limit, which killed a legitimate 255-row, 80-step run at 100 s. Worker errors, exits and unreadable messages reject the run and retire the engine. The console then falls back to the next engine.

In the browser, the single-thread engine runs in its own worker so that a long run never blocks the page. Cancelling terminates that worker and starts a fresh one; messages from a replaced worker are ignored, which prevents a late exit event from rejecting the next run. The threaded engine cancels by stopping and re-initialising its workers. Both remain usable afterwards. The console's Stop button and the presentation drive these paths.

6.5 WebGPU backend

One WGSL compute shader (workgroup size 64, one invocation per cell, eight lanes per invocation) implements all 15 operations behind a switch on a per-row uniform: op, c, p, n, M, birth, survive. Each frame register is a 512 KiB storage buffer. Each row binds a (read), b (read) and o (read_write); the output never aliases an input, by the admission rule in Section 5.3. Each row is one dispatch of 256 workgroups. Rows are encoded into command buffers of 64 steps each. The run executes inside validation and out-of-memory error scopes. Read-back has a limit of 120 s plus 5 ms per row-step, and device loss marks the engine broken.

6.6 Row fusion (1.3.0)

The C engine executes each maximal run of point-wise rows block by block. All rows of the run are applied to 64 cells (2 KiB per register) before the next block, with one barrier per run. Neighbor rows (nsum, life) run alone, each followed by a barrier. Every cell's rows still execute in program order, so results are unchanged. The GPU backend does not fuse rows; this is listed in Section 13.

7. Data formats

7.1 RUSS 1.3 container

Each R file is a RUSS 1.3 tuple-executable, byte-compatible with corpus/tooling/russ_container.py from the RUSSL corpus 2.0.0. All integers are big-endian.

Part	Layout
Header (24 B)	magic 52 55 53 53 0D 0A 1A 0A · major u16 = 1 · minor u16 = 3 · flags u32 = 1 · manifest length u64 (1 B to 1 MiB)
Manifest	Corpus-canonical JSON (Section 7.4)
Records, sorted by path	path length u32 (1 to 1,024) · data length u64 · SHA-256 (32 B) · path (UTF-8) · data

The manifest carries format RUSS.TupleExecutable, version 1.3.0, the payload list (path, sha256, size), payload_root_sha256, tuple_id, hash sha256, container_mode portable and status plume-pipeline-candidate. The tuple has eleven components in fixed order: identity, language, geometry, state, coupling, mechanics, admission, projection, fabric, evidence, execution.

7.2 Payload of every R file

Record	Content
language/main.russl	The RUSSL/1.0 program (Section 7.3)
language/main.russir.json	Its canonical IR, from the RUSSL lowering

Record	Content
state/field.u32le	524,288 B: shape (1, 1, 8, 128, 128) in Penteract order, c6 = lane, c7 = row, c8 = column, little-endian u32 holding 20 bits
state/field_descriptor.json	RUSS.PenteractField/1.0: rank 5, axes c4–c8, dtype u32, levels 20, population identity5
state/pyramid.json	PlumeLattice.Pyramid2/1.0: SHA-256 of the field mod 2^j, j = 1…20
evidence/provenance.json	Stage, XPU, law, steps, input tuple id, SHA-256 of the LCUF-A program, compiler and console versions
mssl/master_coupled_mechanics.mssl · mssl/final_admission.mssl	Digest-bound MSSL modules from the corpus golden fixture

The engines use chunky order and the container uses planar order, and the reader converts between them. A container is about 551 KB, of which 512 KiB is the field. The engine is deliberately excluded from provenance, so the same computation on any backend yields the same tuple id. On the operator's AMD GCN-5 machine, R2 tuple ids were identical between 8 Wasm threads and WebGPU at 40 + 40 steps (bd36306e…), and between WebGPU and single-thread Wasm at 5000 + 5000 steps (6935b16b…).

7.3 The 20-layer RUSSL program

R0, R1 and R2 each carry a RUSSL/1.0 unit (plume.r0, plume.r1, plume.r2) of 27 rows in three frames:

Frame	Rows
F0000 construct-evaluate	FD field.frame (shape 1,1,8,128,128; sample u32) · FP field.frame.candidate · PY field.pyramid (levels 20; downsample mod2pow)
F0001 pyramid	L001–L020: FS field.levelNN, selector mod2pow:j
F0002 evaluate-admit	CD coupling.moore · ME mechanics.result (MSSL master) · AE admission.result (MSSL final_admission) · ER receipt

The IR ids are russir:sha256:1e4ea277… (R0), 9db98755… (R1) and a0cb5d4e… (R2). The JS and Python lowerings give the same ids.

7.4 Identities and canonical JSON

Canonical JSON is the corpus's legacy form: json.dumps with sorted keys, separators "," and ":", and ensure_ascii false. The JS implementation sorts keys by Unicode code point rather than UTF-16 code unit, and refuses integers outside the safe range, so that it matches Python byte for byte. Identities are domain-separated SHA-256 of tag ‖ 0x00 ‖ payload:

```
\mathrm{root} = H(\texttt{RUSS-PAYLOAD/1.0} \,\|\, 0 \,\|\, \mathrm{canon}
(\mathrm{payload\ list})), \quad \mathrm{tuple\_id} = \texttt{russ:sha256:}
H(\texttt{RUSS-TUPLE/1.0} \,\|\, 0 \,\|\, \mathrm{canon}(\mathrm{tuple}))
```

The IR id uses tag RUSSIR/1.0 over the IR without its ir_id. The source hash uses tag RUSSL-SOURCE/1.0 over the canonical source.

7.5 Run log

The console records every action as BFTL/1 steps (PLAN, TOOL, OBSERVE, EMIT) in canonical form. The log can be copied, and the operator's 1.2.0 log was the source of the reference measurements in Section 10.

8. Verification architecture

Release 1.2.0 removes Python from every user-facing path. Python remains only in the test suite, as an independent second implementation to compare against.

8.1 Components

Module	Port of	Function
src/russl.js	russl_reference.py: canonical_source, parse_russl, lower_russl	Source canonicalisation (BOM, CR/LF, NFC, trailing blanks, forbidden controls), the full reference-profile parser with its 32 operators and 28 aliases, closed argument keys, control-ancestry dependencies (BigInt bitsets), MSSL binding rules, effect checks, and lowering to canonical IR. Error codes match the reference (RUSSL-E-PARSE-001 …).
src/schema.js	schema_subset.py	The supported JSON Schema keyword subset; unknown keywords are errors and none is skipped
src/schemas.js	corpus machine/schemas	RUSS tuple manifest, RUSSIR and Penteract field schemas, embedded
src/russ.js	russ_container.py	Container writer and verifying reader

8.2 Checks made on every load

readRuss performs, in order:

1. Structural checks: size ⩽ 64 MiB, magic, version, flags, manifest bounds, and strict JSON. Strict JSON means UTF-8, no duplicate keys, well-formed Unicode in every string and key, finite numbers, at most 100,000 values and depth 64. The manifest must also be canonical and match the manifest schema.
2. Every record: exact entry keys; safe paths (no empty, . or .. segments, no trailing dot or space, no Windows device names); no case-colliding duplicates; bounds; and the path and SHA-256 matching the manifest. Records must be sorted, with no trailing bytes.

3. Payload root, tuple arity and order, identity/geometry/state profile, tuple id, fabric root, the state and authorisation boundary, and the allow-list boundary (arbitrary_code false).
4. Language: the IR matches the RUSSIR schema; ir_id and source_sha256 recompute; and **the .russl source is re-lowered and must equal the stored IR exactly**.
5. Mechanics: modules are unique, each binding matches its module digest, no module is unbound, and the admission policy is bound to an ADMISSION_EVAL row.
6. Geometry: the descriptor matches the field schema; descriptor axes equal geometry axes; exactly one FIELD_DEFINE matches shape, axes and dtype; and the field is 1,1,8,128,128 u32.
7. Content: the 20 pyramid digests recompute from the field. Words above bit 19 are masked to 20 bits and reported as a warning.

A file with any failure is not loaded as R0. tools/verify_r.mjs runs the same checks from the command line on the portable Node runtime, with no Python.

8.3 Parity with the reference verifier

Each of the following has a crafted container with all digests, the payload root and the tuple id recomputed, so that only the targeted rule can catch it. Both implementations refuse all 15:

- identity profile changed
- arbitrary_code true
- duplicate JSON key in the IR
- duplicated mechanics module
- geometry axes differing from the descriptor
- a payload path named constructor
- a null FIELD_DEFINE argument
- an edited IR with every id recomputed (caught only by re-lowering)
- an extra manifest key
- hash "md5"
- a lone-surrogate status
- a lone-surrogate provenance string
- provenance with 100,001 values
- descriptor sparse "yes"

- an admission policy not bound to ADMISSION_EVAL

The JS lowering also matches the Python lowering on every corpus source. Section 11 gives the counts.

9. Security and hardening

9.1 Threat model

The console runs locally, on loopback or from a file, or as a sandboxed artifact page. It processes three kinds of untrusted input: .russ files supplied by the operator or a third party, LCUF-A source typed or pasted into the editor, and HTTP requests that reach the local server. The design goals are no hang, no crash, no out-of-bounds memory access, and no silent acceptance of a malformed file. The system makes no confidentiality claims. Containers carry no secrets and are not signed: SHA-256 identities detect change but do not authenticate an author.

9.2 Controls

Surface	Control
.russ input	Every check in Section 8.2. A structural failure throws RussError; nothing is loaded on any failure. Files over 64 MiB are refused before reading.
LCUF-A source	Length ⩽ 65,536 characters. Ring and argument digits are bounded before the primality test, which therefore cannot be made slow. Reserved register names are refused, so object-prototype collisions are impossible. Static admission is described in Section 5.3.
Engine requests	Shared admission check (Section 6): words ⩽ 20 bits, 1 ⩽ steps ⩽ 100,000, exact frame size. The native engine re-validates the ring, every instruction, the phase counts and the thread arguments, and refuses with 0.

Surface	Control
Liveness	A progress watchdog on threads; cancellation on both CPU engines; a read-back limit and device-loss detection on WebGPU; worker error, exit and messageerror all handled. Every busy section releases in finally, and controls stay locked during start-up.
Page	Engine and error text is HTML-escaped. Programs, step counts and R0 are snapshotted at run start, and a generation counter refuses to finish a run whose R0 changed. Edited programs and step counts mark R1 and R2 stale.
serve.mjs	Binds to 127.0.0.1 only; GET and HEAD only. Paths are resolved and contained with a separator-aware prefix check. Malformed percent-encoding and NUL bytes return 400. Only known extensions are served, as regular files. Responses carry nosniff, no-referrer and no-store; client errors drop the socket.

9.3 Defects closed by the hardening

Between 1.1.0 and 1.2.0, the two independent reviews found that:

- the server crashed on /%E0%A4%A;
- readRuss threw raw RangeError and TypeError on truncated files;
- a worker death hung the threaded engine;
- R0 could change mid-run;
- the JS verifier accepted eight classes of file that Python refuses;
- the C engine's phase counts could wrap in u32, and nt = 0 divided by zero;
- the 1.2.0 bundler, during development, dropped all but the first name of multi-name exports, so the built page failed at start-up.

All are fixed, and each fix has a regression test. The bundler failure is now caught by a browser smoke test at the end of every build.

10. Performance

10.1 Method

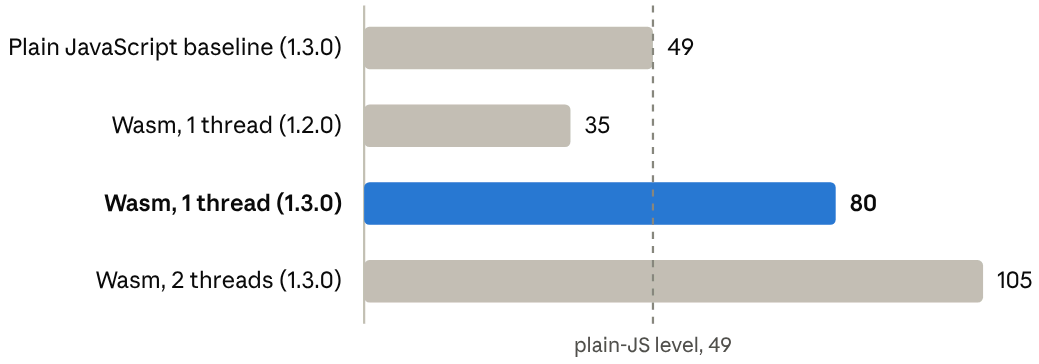
Throughput is reported in **lane-updates per second**: the frame holds W = 8 lanes × 128 × 128 cells = 131,072 twenty-bit words, and one step of a phase updates every word once, so rate = 131,072 × steps / wall time. The console times each phase from dispatch to the moment the engine reports completion (`mlu(steps, ms)` in `page.html`); for every engine the input upload happens before the clock starts; for WebGPU the clock stops only after the result buffer has been mapped back to host memory, so GPU figures include read-back, while the CPU engines work in place in linear memory and have none. Every timed run is also a correctness run: the digest of the final field is compared with the reference digest, and a timing whose digest differs is reported as DIFFERS and not counted. The presentation's race uses the same law, the same seed and the same step count for every engine, including a plain-JavaScript baseline written as ordinary typed-array loops, so that each claim is relative to code a programmer would write without the XPU.

Two machines appear below. The **build host** is the cloud container in which both releases were built and tested (software WebGPU only, so its GPU figures are not reported). The **reference machine** is the operator's Windows workstation (AMD GCN-5 graphics, 8 hardware threads), measured from the BFTL/1 log the operator supplied for release 1.2.0.

10.2 Measurements

One 1.3.0 Wasm thread runs BF1 at 80 M lane-updates/s, 2.3× the 1.2.0 engine

BF1 (2-adic, 131,072 lanes per step), build host, millions of lane-updates per second; longer is faster



Two-thread runs ranged 105–118; the lowest is drawn. The 1.2.0 thread fell below plain JavaScript; the 1.3.0 thread clears it by 1.6×.

CHANGELOG 1.3.0 and build-host race runs, 2 November 2026 · BF1 only · the same digest from every engine

On the build host the 1.3.0 CPU engine is the first in which a single Wasm thread outruns the plain-JavaScript baseline. BF2 rose less, from about 9 to 13 M/s on one thread, because division by 5^8 still dominates its digit arithmetic (§10.3).

Reference machine, 1.2.0	BF1 (M lane-updates/s)	BF2 (M lane-updates/s)
WebGPU, AMD GCN-5, 5,000 steps	116.7	119.0
Wasm, 8 threads	35.5	14.8
Wasm, 1 thread	15.8	5.0

On the reference machine the GPU leads both laws and is nearly indifferent to the ring: its BF2 rate equals its BF1 rate within 2 %, whereas the CPU loses a factor of 2.4–3.2 moving from $Z/2^{20}$ to $Z/5^8$. Eight threads give 2.2× (BF1) and 3.0× (BF2) over one, well short of linear; the per-step barrier and memory bandwidth, not arithmetic, bound the threaded engine at this frame size. The reference figures predate 1.3.0, and the 1.3.0 CPU gains have not yet been measured on that machine.

10.3 The 1.3.0 optimisation, analysed

A per-operation profile of one BF1 step on the 1.2.0 engine attributed **2.683 ms of roughly 3.7 ms** to the `life` op. The cause was the rule lookup `(survive >> live) & 1`: a shift by a per-lane variable amount has no WebAssembly SIMD instruction, so clang scalarised the whole loop body. Release 1.3.0 replaces the lookup with nine compare-and-select steps, `res |= (live == q) & (alive ? S[q] : B[q])` for q = 0 … 8, which uses only lane-wise compare, and and or, and is exact for every B/S rule, not only B3/S23. The same profile then gives **0.933 ms** for `life`, a 2.9× reduction in that op and the bulk of the 2.3× gain at the step level.

Three smaller changes contribute. Ring parameters (p, n, modulus, mask) are copied from linear-memory globals into locals before each loop, because loads from memory that may alias the output defeated the vectoriser. Consecutive point-wise rows are fused and executed block by block over 64 cells, so their intermediate registers stay in cache and the threaded engine meets the barrier once per fused run rather than once per row. For odd p, digit addition reads a $p^2 \times p^2$ table (4,096 entries, 8 KiB) that adds two base-p digits at a time, halving the divisions. An arithmetic select variant for the Life rule, tried for p = 2, was slower than compare-select and was reverted.

None of these changes alters a result. Every 1.3.0 engine reproduces the 1.2.0 tuple identifiers and field digests, and the suites in §12 compare each engine with the NumPy reference bit for bit.

11. Verification evidence

11.1 Automated suites

Both releases carry the same six suites. All pass on the build host for each release's commit, and the 1.3.0 engine changes altered no expected value.

Suite	Cases	Independent oracle
test/xpu.test.mjs	36	NumPy implementations of BF1 and BF2 (test/ref_numpy.py); the JS reference interpreter
test/russ.test.mjs	13	Round-trip, rebuild reproducibility, hostile structure
test/russl.test.mjs	28	Python russl_reference.lower_russl (12 corpus examples, 3 templates, 6 invalid vectors, 7 mutations)
test/hostile.test.mjs	15	Python russ_container.verify_file with the corpus schemas
test/page.test.cjs	13	The built console in Chromium with a software WebGPU adapter
build.sh smoke test	1	The built page must start and finish a pipeline with no page errors

The xpu suite covers:

- bit-exactness of both stages and the chain;
- the JS reference against the native engine;
- compiler diagnostics;
- constant reduction across the CPU engines;
- refusals of bad steps, frames, rings and instructions;
- overlapping queued runs;
- a worker killed mid-run, failing in about 200 ms;

- a silently dead worker, caught by the watchdog in about 1.9 s at a 1.5 s threshold;
- a multi-second run that the watchdog must not stop;
- cancellation and recovery on both CPU engines.

11.2 Cross-backend and cross-machine agreement

- **Build host:** single-thread Wasm, threaded Wasm and WebGPU (SwiftShader) gave the same R2 field digest, 7500a59c3b3bf515, for seed 7, density 35 %, 40 + 40 steps. The hologram check held on WebGPU.
- **Operator machine** (AMD GCN-5 GPU, 8 CPU threads, console 1.2.0): Compare engines reported the same digest, 7500a59c3b3bf515, on three engines. Hardware, operating system and GPU all differ from the build host. Tuple-id agreement at 40 and 5000 steps is recorded in Section 7.2.

11.3 Independent reviews

Two reviews were run by separate agents that had not seen the author's reasoning. Both reviews confirmed each finding with a probe script before reporting it.

Review	Target	Findings	Most severe	Outcome
1	1.1.0 development tree	6	The threaded engine hung on overlapping runs or a dead worker	All fixed in 1.1.0
2	1.1.0 release	9	The development 1.2.0 bundle failed at start-up; start-up clicks locked the console	All fixed in 1.2.0

The second review also compiled the WGSL kernel in Chromium's WebGPU (Tint) without errors. It matched the reference on BF1, BF2 and 60 random programs, and 120 random programs across 8 rings showed zero C-versus-reference differences.

11.4 Exhaustive and randomised mathematical checks

Iterating T on $Z/2^{20}$ and f on $Z/5^{8}$ confirmed both are single-cycle (Section 4). A randomised pair test confirmed that f is 1-Lipschitz. Plume tests from bits 0, 5 and 12 never touched lower bits and stayed within radius t.

12. Release 1.3.0 additions

12.1 Plume XPU Live

Plume XPU Live (web/show.html) is an eight-slide presentation that runs the release's own engines on the viewer's machine. It shares the console's bundle; nothing on it is pre-recorded.

Slide	What runs live	What it demonstrates
1 Title	Engine detection; a 200-step warm-up on the fastest engine	The viewer's hardware and measured throughput
2 Cell	BF1 one step per animation frame; the 8 × 20 bit grid of one cell	The Pixel160 state and the Life plane
3 Plume	Two frames one bit apart, 64 steps	The cone bound of Section 4.6, checked every step
4 Pipeline	R0 → BF1 → R1 → BF2 → R2 at 400 steps per stage; each R built and re-verified	Sealed, chained files
5 Race	The same 400 BF1 steps on every engine in turn, in 40-step chunks; bar length is measured speed	Throughput against a baseline, and identical digests
6 Hologram	BF1 on R0 mod 2^j for all j = 1…20	The hologram theorem at every layer
7 Trust	The verifier on the intact R1, one flipped field bit, one edited RUSSL character, and one truncated byte	Refusal with the exact reason
8 Scoreboard	This machine's race results beside the operator's 1.2.0 reference run; Copy as CSV	The figures, labelled by version and machine

Chunking the race is exact for BF1: from2 and to2 are identities for p = 2, so ten 40-step runs equal one 400-step run. The race checks this, because every engine must reproduce the same digest.

12.2 Plain-JavaScript baseline

The race includes the BF1 law written directly as typed-array loops in a worker, with no LCUF-A engine. It produces the same digest as every engine. It is the outside baseline for every speed claim: what a programmer would write without the XPU.

The baseline exposed a weakness that 1.2.0 had not shown. On the build host it ran at 49 M/s, faster than the 1.2.0 single-thread engine at about 35 M/s. Section 10.3 analyses the cause and the fix. After the fix, one engine thread is about 1.6× the baseline and two threads about 2.4×.

12.3 Other changes

- "Start Plume XPU Live.cmd" opens the presentation in JA21 Blank Console through the isolated local server, so threads and the GPU are available.
- The CPU-engine changes are described in Sections 6.6 and 10.3; LCUF-A compiler 0.3.2 records them.
- Both repositories carry LICENSE (GNU GPL v3, unmodified), COPYRIGHT and NOTICE, and every source file carries an SPDX GPL-3.0-only header.

13. Known limitations and open issues

#	Limitation	Effect	Proposed resolution
L1	BF2 on the CPU is bound by integer division by the runtime modulus	About 13 M/s on one thread on the build host, against about 80 M/s for BF1	Reciprocal (Barrett) reduction with precomputed inverses for p, p^2 and M
L2	The GPU backend issues one dispatch per row and does not fuse rows	Fixed overhead dominates short runs (68 M/s at 40 steps against 117 M/s at 5000 steps on the reference GPU)	Generate a fused WGSL kernel per run of point-wise rows
L3	Thread scaling is sub-linear (about 2× at 8 threads in the 1.2.0 reference run)	Barrier per row; memory traffic of separate passes in 1.2.0	Re-measure with 1.3.0 fusion; widen bands; fuse neighbour rows

#	Limitation	Effect	Proposed resolution
L4	The 2→5→2 re-encoding discards about 2.7 bits of each upper word (Section 4.9)	R2 is not invertible to R1	Inherent to crossing the rings; a 10-digit 5-adic word ($5^{10} \approx 2^{23.2}$) would hold all 19 upper bits without loss but exceeds the current 2^{21} ring bound
L5	Unicode version differences: JS NFC normalisation and \p{Cc}/\p{Cf} classes follow the browser's ICU; Python follows its unicodedata	Possible disagreement on unassigned or newly assigned code points	Pin a test vector set of edge code points
L6	The JS storage-leak pattern uses an ASCII word boundary; Python's is Unicode-aware	A theoretical difference on non-ASCII text adjacent to ".zarr"	Use the Unicode-aware boundary or an explicit class
L7	JSON numbers: JS cannot distinguish 1 from 1.0	JS is stricter where Python accepts a float that canonicalises to "1.0"	Accepted: the failure direction is refusal
L8	Schema pattern $: Python's re.search lets $ match before a trailing newline	A digest with a trailing newline passes Python's schema check but not the JS one	Accepted: the failure direction is refusal; digest checks elsewhere are exact
L9	The published artifact page cannot use threads (no cross-origin isolation) and cannot save files	Single-thread and GPU only when viewed on claude.ai	Run locally through the bundled server
L10	WebGPU correctness is evidenced on SwiftShader and one AMD GCN-5 GPU	Other drivers are untested	Run Compare engines on each new GPU

#	Limitation	Effect	Proposed resolution
L11	No conformance claim against the full RUSSL corpus (259 of 293 requirements NOT_RUN in the corpus itself)	Profile-level interoperability only	Out of scope for these releases
L12	Containers are content-addressed but not signed	Integrity, not authenticity	A detached signature over tuple_id
L13	Repositories are prepared and committed but not yet pushed: the session lacked GitHub App access	Public availability pending	Operator push, or GitHub App installation

14. Reproducibility

14.1 Toolchain (build host)

Component	Version
C compiler	Ubuntu clang 18.1.3, target wasm32, wasm-ld 18
Node.js	22.23.4 (the same version as the portable runtime shipped with bytefoundry-0.99.2-win64)
Python (tests only)	3.13, NumPy 2.5
Browser automation (tests only)	Playwright 1.63 with Chromium; software WebGPU via SwiftShader
Host	2 vCPU Intel Xeon @ 2.10 GHz, Linux

14.2 Build and test

```
./build.sh                                  # wasm (1 thread and threaded),
web/index.html, web/show.html, browser smoke test
python3 test/ref_numpy.py                   # NumPy reference frames
node test/russ.test.mjs                     # writes test/out/r0..r2 used by
later suites
node test/xpu.test.mjs
node test/russl.test.mjs  <RUSSL-Zarr-main/corpus>
node test/hostile.test.mjs <RUSSL-Zarr-main/corpus>
node serve.mjs & node test/page.test.cjs
node tools/verify_r.mjs test/examples/*.russ # Python-free verification
```

14.3 Artefact identities

Artefact	1.2.0	1.3.0
Commit	e77910507184a9dee3f60dfc11d4b79a99e72324	958485eafda3b34e14c23892be19214958193c83
Tag	v1.2.0	v1.3.0
Repository	LKVexa/Blank-MIMIC-CIMIM	LKVexa/MIMIC-CIMIM-Demonstration
xpu.wasm SHA-256 (prefix)	3ecd1f6c92abc594	41ba8f680b14928d
xpu-mt.wasm SHA-256 (prefix)	b0c409bd6caf6c71	c67ea14ee5f5b970
Files committed	43	46

14.4 Determinism

Given the same input field, program, step count and rule, every engine yields the same output words, and R files are reproducible: rebuilding R1 gives the same tuple id. Example containers ship in test/examples:

- R0: russ:sha256:7f03a575… (seed 7)
- R1: be141817… (40 BF1 steps)
- R2: c674e477… (40 BF2 steps)

Tuple ids include provenance, so the same field labelled with a different console version or step count has a different id. The field digests in Section 11.2 are version-independent.

15. Licensing and provenance

Both releases are Copyright (C) 2026 RUSSELL PHILIP SMITHSON and licensed under the GNU General Public License, version 3 only (GPL-3.0-only). Each repository carries LICENSE (the unmodified GPL v3 text, SHA-256 3972dc97…), COPYRIGHT and NOTICE. Every source file has an SPDX-License-Identifier: GPL-3.0-only header with the copyright line, and the built pages carry the notice as an HTML comment.

Component	Origin	Terms
Engine, engines, compiler, container code, console, presentation, server, tools, tests	Original work of the copyright holder	GPL-3.0-only
src/russl.js, src/schema.js	Ports of russl_reference.py and schema_subset.py, RUSSL Research & Institutional Corpus 2.0.0	GPL-3.0-only
src/schemas.js	The corpus machine/schemas	GPL-3.0-only
MSSL modules (web/mssl, and copies inside templates and example containers)	Corpus golden fixture	GPL-3.0-only
AGNT frame and BFTL/1 conventions	ByteFoundry	MIT (compatible)
Fonts (IBM Plex, Archivo)	Google Fonts, loaded at run time	Not redistributed

lcufa.js was offered as "MIT OR GPL-3.0-only" in 1.0.0–1.3.0 development and is GPL-3.0-only in the released repositories. The software comes with no warranty (GPL v3 sections 15 and 16).

16. References

1. A. Klimov and A. Shamir, "A New Class of Invertible Mappings," in *Cryptographic Hardware and Embedded Systems – CHES 2002*, LNCS 2523, Springer, 2003, pp. 470–483. doi:10.1007/3-540-36400-5_34

2. M. V. Larin, "Transitive polynomial transformations of residue class rings," *Diskretnaya Matematika* 14:2 (2002), 20–32; English translation *Discrete Mathematics and Applications* 12:2 (2002), 127–140. Math-Net.Ru dm238
3. V. Anashin, A. Khrennikov and E. Yurova, "T-functions revisited: new criteria for bijectivity/transitivity," *Designs, Codes and Cryptography* 71:3 (2014), 383–407. arXiv:1111.3093
4. W3C, *WebGPU Shading Language*, Working Draft, 12 January 2023 (A. Baker, M. O. Derin, D. Neto, eds.). w3.org/TR/2023/WD-WGSL-20230112
5. E. R. Berlekamp, J. H. Conway and R. K. Guy, *Winning Ways for Your Mathematical Plays*, vol. 2, Academic Press, 1982 (construction of a universal computer in Conway's Life).
6. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015 (SHA-256).
7. RUSSL Research & Institutional Corpus 2.0.0 (LKVexa and contributors, GPL-3.0-only): WP01 Language Specification, WP04 Penteract Field Specification, tooling/russl_reference.py, tooling/russ_container.py, tooling/schema_subset.py, machine/schemas. Local copy, RUSSL-Zarr-main/corpus.
8. R. P. Smithson, *BFTL/1: The ByteFoundry Transcript Language*, revision 2, 2026-11-02 (ByteFoundry 0.101.0).
9. R. P. Smithson, *ByteFoundry 0.99.2*, src/lib/byteproc (frame.ts, russ.ts, lcuf.ts, lcufx.ts), MIT.
10. Plume Lattice: Interfacial Computation over Pixel160 / Byte20 Frames (design note, 2 November 2026), the source of the BF1 law and the hologram property.

Items 1–4 were checked against the linked pages on 2 November 2026. Items 5 and 6 are standard references cited bibliographically. Items 7–10 are the operator's own materials.